2019 IEEE Intelligent Vehicles Symposium (IV)
Paris, France. June 9-12, 2019

# Semantic Segmentation on Automotive Radar Maps

Robert Prophet, Gang Li, Christian Sturm, and Martin Vossiek

***Abstract*— As radar sensors can measure an object's range and velocity with a high degree of precision, moving objects can be successfully classified, as well. Classifying stationary objects still needs a lot of research, however. In this paper, we use popular semantic segmentation networks in order to classify the vehicle's immediate infrastructure. To this end, a full 3D measurement is performed with a test vehicle equipped with four high resolution corner radar sensors. A preprocessed point cloud is transformed into various radar maps for input to a neural network. Simulations as well as real-world measurements show an overall intersection over union of 84 and 77%, respectively, as well as an overall accuracy of 95 and 90%, respectively, being a new benchmark for this young research field.**

## I. Introduction

Radar sensors generally measure an object's range and velocity more accurately than camera systems. Moreover, since radar as an active sensor is more robust to changing weather conditions, it is well suited for the automotive field. However, in terms of object classification, especially of static objects, the radar sensor's performance still falls significantly short of expectations.

A lot of recent papers concern the classification of moving objects. In [1], various machine learning-based classifiers, which are fed by engineered features, are used to distinguish between pedestrians and three other kinds of road users. The same procedure is used in [2], but instead of engineered features a feature descriptor is applied. The authors in [3] use a semantic segmentation network on radar point clouds in order to classify five different types of road users. The advantage of this approach is that it does not require clustering algorithms or feature engineering.

The static environment must be classified for path planning – a key element of autonomous driving. In this case, the radar sensor cannot make use of the characteristic Doppler signature of an object, making the classification process more challenging. In [4], a radar-based occupancy gridmap (OGM) is used to identify various infrastructure objects, such as buildings, cars, and fences. Specifically, potential objects are extracted by an externally connected component analysis system and subsequently input to a deep convolutional neural network. This approach achieves classification accuracies between 60 and 80% for most classes. Instead of the clustering step, the authors of [5] use the fully convolutional neural network FCN introduced in [6] for semantic segmentation, to distinguish pixel by pixel between cars and other objects inside the OGM. With this method, the authors achieve an overall accuracy of 80% and slightly above it depending on certain parameters.

In this paper, we also use semantic segmentation networks in order to avoid the clustering step being a bottleneck for the algorithm's performance. Besides FCN, we use the popular semantic segmentation networks SegNet and U-Net to distinguish between parked cars, curbstones, and other infrastructure. Both architectures were introduced in 2015 [7, 8]. SegNet, especially, has important advantages over FCN. We show that our algorithm achieves better results than existing approaches. To accomplish this, we use two further radar maps in addition to the OGM, which we shall discuss in the next section.

## II. Radar Maps

As, in contrast to camera data, there is currently no public radar data set available, we use a custom data set for our trials. This data set was created by four radar sensors mounted at the corners of the test vehicle at an angle of 50 deg to the vehicle's longitudinal direction for each sensor. These Valeo radar

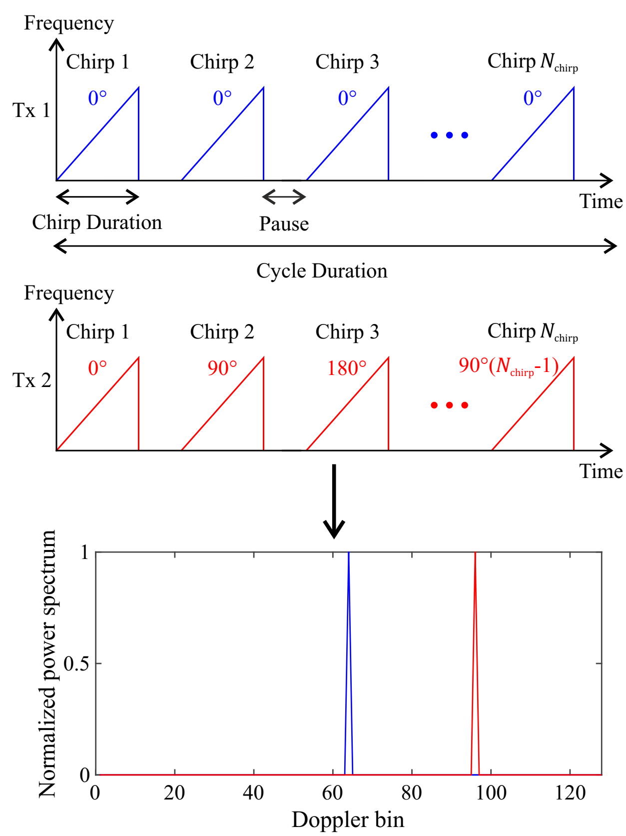


Fig. 1. The two top diagrams show the Doppler-multiplex modulation with a 90 deg phase increment. The lower diagram is the resulting Doppler spectrum (zero-frequency component at the center) for one static detection.

R. Prophet and M. Vossiek are with the Institute for Microwaves and Photonics at the Friedrich-Alexander-University Erlangen-Nuremberg, 91058 Erlangen, Germany (e-mail: robert.prophet@fau.de).

G. Li and C. Sturm are with Valeo Schalter und Sensoren GmbH, 74321 Bietigheim-Bissingen, Germany.

sensors are based on fast-chirp modulation at $f_c = 79$ GHz. They operate with $B = 1.6$ GHz bandwidth and 9.4 cm range resolution for a maximum range of $R_{\max} = 24$ m. Each sensor features four receivers and two transmitters. The so-called Doppler-multiplex approach introduced by Sturm in 2018 [9] is applied to enable the receivers to distinguish the signals radiating from the different transmitters. In the case of two transmitters, the second transmitter changes its carrier frequency phase by 90 deg from one chirp to the next, while the first transmitter keeps its phase constant. The transmitters are separated in the range-Doppler matrix by the virtual Doppler frequency produced by this procedure. Fig. 1 clearly exemplifies this concept for 128 Doppler bins. Both azimuth and elevation are estimated with the virtual antenna array consisting of eight receivers.

The time signal for each sensor is transformed to a 4D spectrum proportional to range, Doppler, azimuth, and elevation. Detections in this spectrum are identified by means of a constant false-alarm-rate algorithm. While range and velocity can simply be calculated by linear equations, the angle estimations require more sophisticated techniques, such as beamforming algorithms. Finally, each radar sensor outputs a detection list containing range $R$, relative radial velocity $v_D$, azimuth $\alpha$, elevation $\varepsilon$, and signal to noise ratio (SNR) $P$ for each detection. The SNR describes the ratio of the magnitudes of the corresponding detection's element of the 4D spectrum and its neighboring elements. This is a measure of the reflected power of the obstacle. At the end of these preprocessing steps, the spherical coordinates are transformed into a global Cartesian coordinate system.

Moving detections are identified by comparing $v_D$ and the corresponding sensor velocity, which is calculated from odometry data from the test vehicle. We ignore these detections as we only want to classify the static environment. Even current automotive radar sensors make relatively few detections in one sampling loop due to the limited sensor angle resolution. Hence, it is beneficial to accumulate detections over time. These detections are accumulated by calculating the necessary ego motion from one sampling loop to the next by means of the constant turn rate and acceleration (CTRA) motion model [10], fed by the odometry data. A simultaneous localization and mapping (SLAM) algorithm [11] is run if the odometry data is poor. Consequently, a 4D point cloud, namely one dimension for each of the three spatial coordinates $x, y, z$ as well as $P$ ($v_D$ is neglected for static detections as it only contains a component of the ego velocity), represents the information input for all subsequent steps. In the literature, the $x$-$y$-plane is typically parallel to the ground, where $z = 0$.

SegNet and other semantic segmentation networks normally process images using three channels. Therefore, we transform the point cloud into three different one-channel/2D maps as input to the neural network: OGM, SNR Gridmap (SGM), and Height Gridmap (HGM). Since we classify the environment at roughly the same heights as the test vehicle, where $z_{\text{ego}}$ denotes its height over ground, we only use point cloud detections that fulfil $-0.3\text{ m} \leq z \leq z_{\text{ego}} + 0.3$ m. The cross-sections of almost all obstacles in this zone, are only slightly affected by the height (objects suspended in space are rare exceptions). Moreover, this area is suitable for path planning, as the upper limit is greater than $z_{\text{ego}}$.

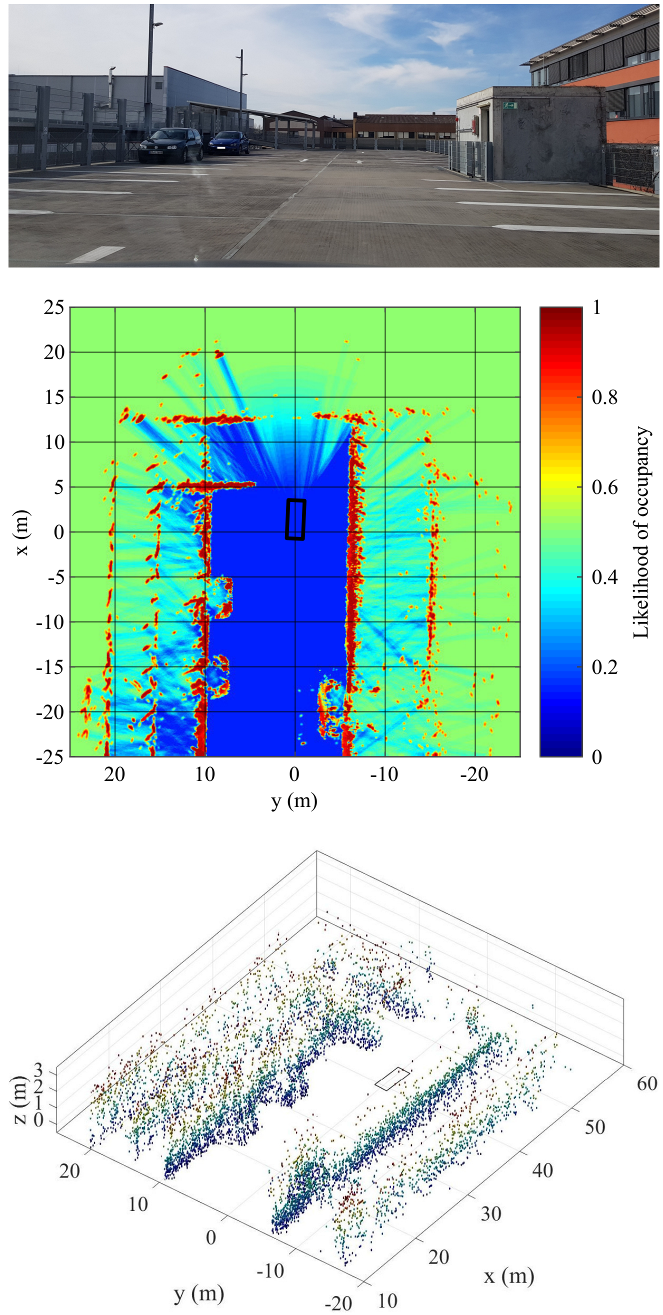


Fig. 2. The top image shows the completely stationary real-life scene. The test vehicle drove straight to the end of the park deck at approx. 15 km/h.

The center image is the 50x50 m OGM, which divides the environment into free (blue) and occupied (red) areas. Unseen areas (green) have the prior probability of 0.5. The test vehicle is generally outlined as a black rectangle. As this OGM contains only detections below the aforementioned height threshold, the front left lane is mainly modeled as free space.

Finally, the bottom image depicts the 3D occupancy grid, where only occupied areas are visible and colored by their height (the warmer the color, the higher the object). To demonstrate the height measurement capacity of the sensor, $z_{\text{ego}}$ was set to infinite for this image. Consequently, the height differences between the fence, the cars, the building on the right, and the roof over the front left lane are evident.

## A. Occupancy Gridmap

OGMs were first described in [12] and adapted for automotive radar systems in [13]. An OGM is essentially a 2D

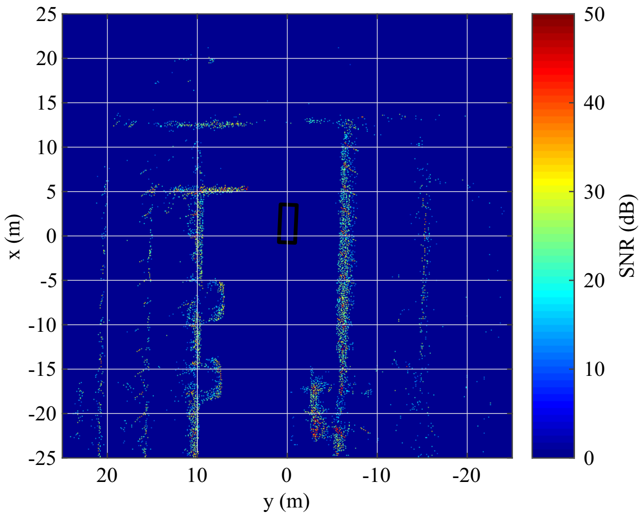


Fig. 3. The image shows the corresponding SGM to the OGM from Fig. 2. Since the environment consists mainly of metallic infrastructure, the reflected power is relatively high and uniform.

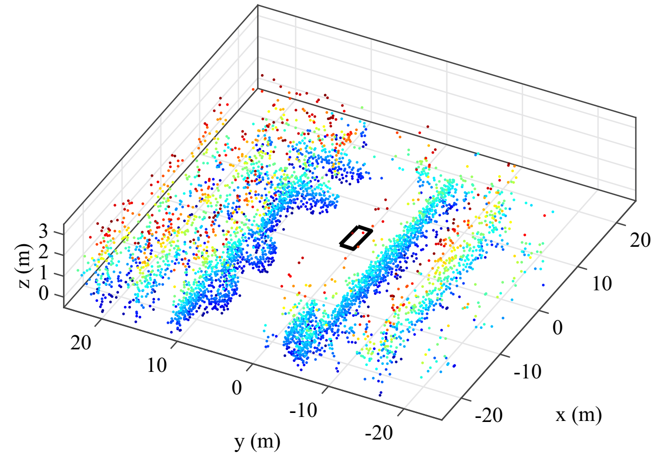


Fig. 4. The image shows the HGM corresponding to the OGM from Fig. 2. Warmer colors indicate greater heights. As mentioned before, and in contrast to octomap, this is not a real 3D presentation, because one horizontal resolution cell contains only one height. However, like the octomap result, the height difference between the two cars and their bordering fence, the roof over the front left lane, as well as the fence on the right and the building behind it, are clearly visible (again, $z_{\text{ego}}$ was set to infinite for this image).

map of free and occupied spaces in the environment. The environment is divided into equally sized cells $m_i$ – this is a common practice with all radar maps – with each cell containing the likelihood of occupancy $p_i$ for the corresponding location with $i = 1, \dots, N_{\text{cell}}$ ($N_{\text{cell}}$ is the total number of cells). We set the cell size to 10 cm, that is, within the sensor's range resolution. 3D occupancy grid approaches improve the scene understanding, but their computational and memory costs are significantly higher. Furthermore, the most popular implementation, namely *octomap* introduced in [14], was mainly created for lidar data, which differ greatly from radar data. Fig. 2 compares a 3D grid built by octomap with an OGM, which we use as the first channel in our neural network.

## B. SNR Gridmap

We feed information on the material-dependent reflective power of the obstacles to the neural network. Specifically, we build a map that contains the SNR $P_i$ for the corresponding cell $m_i$. If two or more detections belong to the same cell, we do not build an average but keep the maximum SNR. This reduces the influence of the direction of arrival of the detection, since this value changes as the test vehicle passes the detection. Fig. 3 depicts the SGM of the scene from Fig. 2.

## C. Height Gridmap

To overcome the disadvantage that we do not use a 3D environmental representation, the third input channel for the neural network provides the height $z_i$ of the obstacle at cell $m_i$. If $n$ detections fall into cell $m_i$, the height is calculated by:

$$z_i = \sum_{j=1}^{n} \left(\frac{z_{i,j}}{R_{i,j}}\right) \cdot \left(\sum_{j=1}^{n} \frac{1}{R_{i,j}}\right)^{-1} \quad (1)$$

This decreases the influence of detections with higher distances, whose measurement uncertainty is greater for trigonometrical reasons. Fig. 4 shows the HGM of the aforementioned scene.

# III. Neural Network Architecture

In a typical classification problem, usually a fully connected layer maps the features detected by convolutional layers to the output class predictions. Doing this, convolutional layers extract semantic information while spatial information is discarded after the pooling layers. At the output layer, all spatial information is lost. For problems where the spatial information is as important as semantic information, a different network architecture must be considered in order to retain the spatial information.

Several semantic segmentation networks have been published in recent years. They all extract semantic information as well as spatial information by assigning every pixel in the input image to a class in the output image. Examples of such networks are FCN, U-Net, DeepLab [15], and SegNet. The advantage of SegNet over other networks is that it is more efficient in terms of both memory and computational time. Hence, it is more suited for real-time applications. It has significantly fewer trainable parameters, since the decoder layers use only the max-pooling indices from the corresponding encoder layers instead of the complete encoder layer. Despite its slimmer architecture, it performs comparably to FCN and U-Net on large datasets [7].

After the input image, SegNet consists of $L$ encoder sections followed by $L$ decoder sections ($L$ is the encoder/decoder depth). The $l$-th encoder section, $l = 1, \dots, L$, features $C_l$ convolution units $E_{l,1}, \dots, E_{l,C}$ that each contain a convolutional layer, batch normalization, and a rectified linear unit (ReLU) as activation function, plus a max-pooling layer with a 2x2 window and stride 2 at the end of the section. The $(L-l+1)$-th decoder features an upsampling layer at the beginning plus $C_{L-l+1}$ convolution units $D_{L-l+1,1}, \dots, D_{L-l+1,C}$ that each contain a convolutional layer, batch normalization, and a ReLU. Note that the upsampling layer of decoder section $L-l+1$ upsamples its input feature map using the stored max-pooling indices from the $l$-th encoder section feature map. Furthermore, the number $noc_l$ of output channels, which correlates with the number of convolution filters, for the $l$-th encoder and $(L-l+1)$-th decoder section is also configurable. Finally, the last decoder output feature map is fed to a soft-max classifier for pixel-wise classification. With

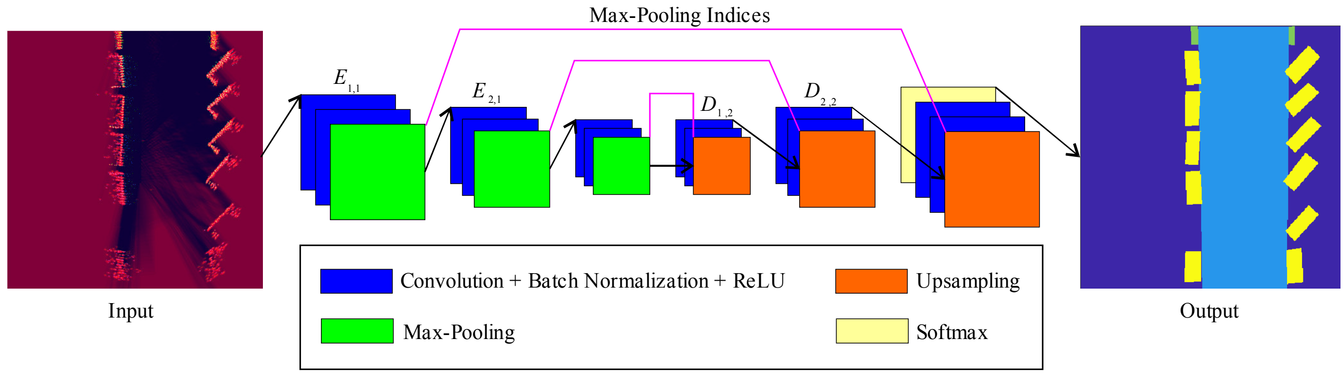


Fig. 5. The figure illustrates the SegNet-2 architecture. The input image consisting of the three superimposed radar maps is fed to three encoder sections. Three decoder sections then upsample the map using the transferred max-pooling indices from the encoder in order to produce feature maps. Finally, the feature maps of the final decoder section are fed to a soft-max classifier, which calculates pixel-wise probabilities of the individual classes. There are four classes in this example: Background (dark blue), Street (light blue), Curbstone (green), and Car (yellow).

$K$ being the number of classes, the output of this soft-max classifier is a $K$ channel image of probabilities.

In this paper, we use the original versions of FCN-8s, which upsamples its final feature map by a factor of 8; a slightly modified original version of U-Net with an encoder/decoder-depth of 4, whose convolutional layers uses padding to retain the data size from input to output; and two SegNet variants. The first SegNet variant – which we shall call SegNet-1 from now on – corresponds to the original architecture. Here, the encoder/decoder depth $L$ is set to five. While the first two encoder, and the last two, decoder sections use two convolution units, the last three encoder, and the first three, decoder sections have three convolution units. The number of output channels follows $\max(2^{5+l}, 512)$ for the $l$-th encoder section and $\max(2^{5+6-l}, 512)$ for the $l$-th decoder section for $l = 1, \dots, 5$. In order to initialize the training process with weights trained for classification on large datasets, the encoder layers are like those in the VGG16 network [16]. Our second variant called SegNet-2 has an even slimmer architecture than SegNet-1 and is shown in Fig. 5. It features an encoder/decoder depth of three and the $l$-th encoder and decoder section own two convolution units with $2^{4+l}$ and $2^{4+4-l}$ output channels for each of them for $l = 1, 2, 3$. This leads to merely 3 MB memory space for the entire network for a 256x256x3 input image.

## IV. Simulation

### A. Dataset

Simulations are used to test new algorithms, as it is difficult to set up test scenarios, like urban driving in a town without any other road users; and real-world experiments are cost intensive. However, the field of complete environmental simulation including radar data generation is still young and requires further development.

In this paper, we use the Matlab Driving Scenario Designer (MDSD) for this purpose, which was first introduced in March 2017. We have defined five environments for this application. The first one is depicted in Fig. 6, which the simulated test vehicle passed at 30 to 50 km/h. The environment consists of streets, curbstones, parked cars, and fences. Note that these actors correspond only roughly to real objects, e.g., cars are modeled as cuboids. Moreover, complex infrastructure objects

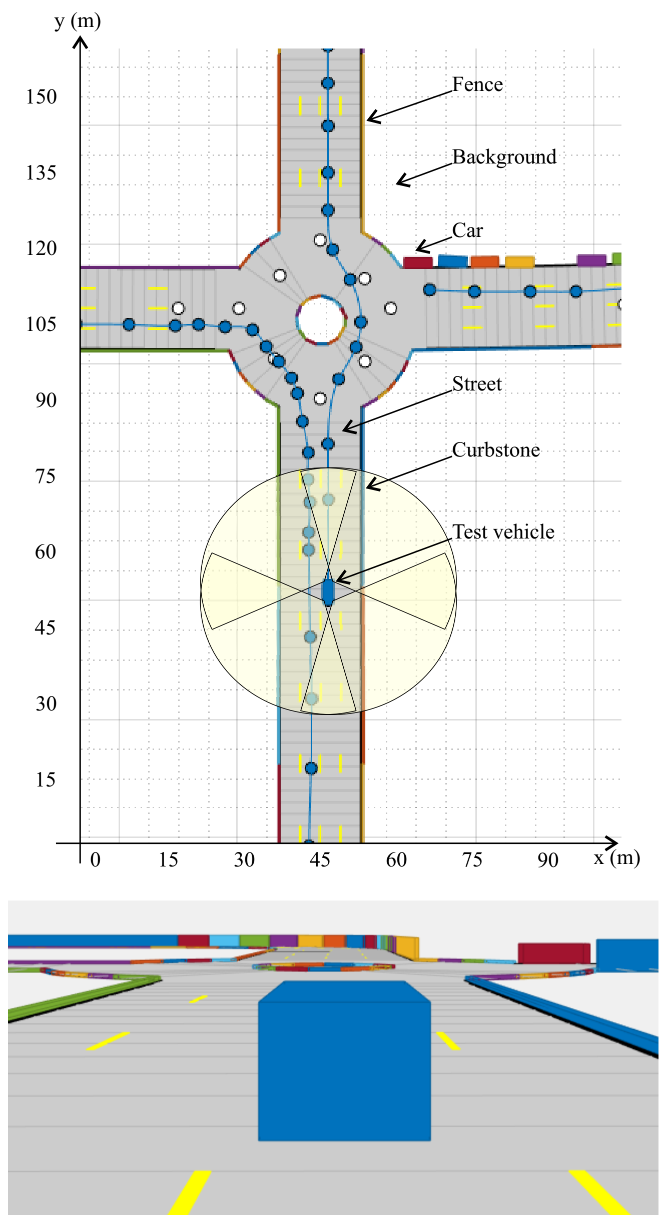


Fig. 6. The test vehicle drives along the blue lane through the simulated environment, where no other road user is moving. The field of view of the sensors is indicated in light yellow.

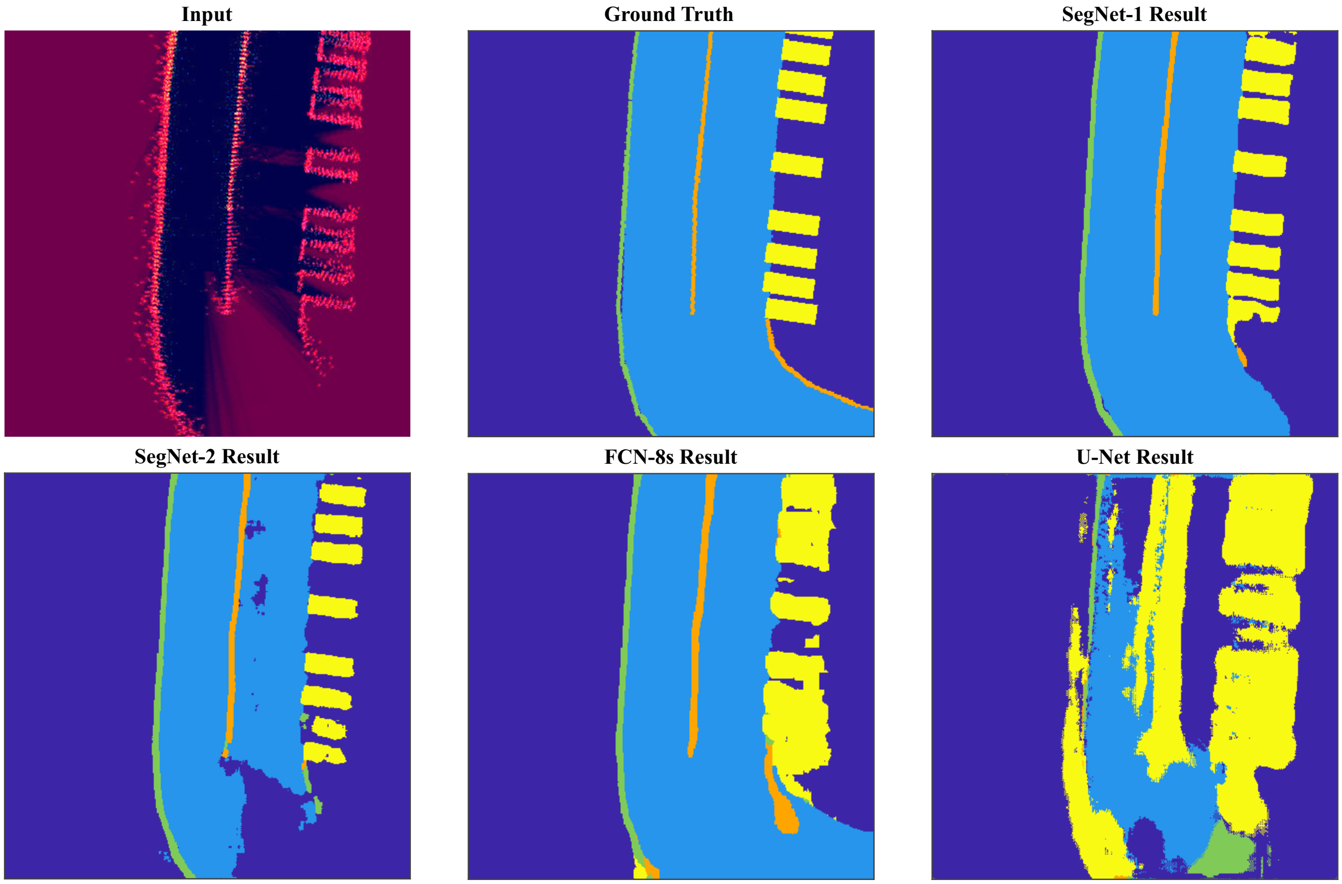


Fig. 7. Comparison between ground truth and the individual results of the networks for a simulated input image. The networks distinguish between Background (dark blue), Street (light blue), Curbstone (green), Car (yellow), and Fence (orange). Again, the test vehicle is in the middle of the map and drove from top to buttom. Due to the limitation of the number of detections per sampling loop, some far objects were not detected yet.

like vegetation and reflecting streets cannot be modeled with the MDSD. The simulated test vehicle was equipped with radar sensors in the same way as the real test vehicle. The MDSD supports configuring sensor noise as well as false alarms, so that we roughly equalized the simulated sensors' responses to the real ones. This includes also a limitation of the number of detection per sampling loop, which we have set to 80 detections per sensor. However, we cannot match the real sensors perfectly, as certain parameters are not configurable, e.g., the angular uncertainty is constant over the whole field of view, whereas the real sensor features an angular uncertainty that is a function of the angle.

The dataset consists of 3140 images sized 384x384 pixels, which is split into an 80% training and 20% test dataset. Each image corresponds to a clipping of the global map, so that the center of the rear axle in the test vehicle is always in the center of the map (pixels (193;193)). This is done, since only the immediate vicinity of the test vehicle is inside the sensors' fields of view. Consequently, with a cell size of 10 cm, each image represents 38.4x38.4 m around the test vehicle (the middle of the pixels lie on the grid points). As the maximum sensor range is 24 m, almost the complete image is visible for the radar sensors. Furthermore, we rotate every map once in order to keep the test vehicle's orientation inside the images constant from top to bottom. Doing this, the passed area always lies mainly in the upper half of the image. The labeling mirrors the created environment. Pixels are assigned to the five classes Background, Street, Curbstone, Car, and Fence.

TABLE I. NETWORK EVALUATION FOR SIMULATION DATA

| **Network** | | **Metrics** | | |
|---|---|---|---|---|
| ***Name*** | ***Class*** | ***Acc*** | ***IoU*** | ***BF*** |
| SegNet-1 | Background | 0.991 | 0.985 | 0.963 |
| | Street | 0.984 | 0.981 | 0.974 |
| | Curbstone | 0.978 | 0.696 | 0.965 |
| | Car | 0.915 | 0.831 | 0.919 |
| | Fence | 0.903 | 0.697 | 0.950 |
| SegNet-1 (upper image half) | Background | 0.991 | 0.981 | 0.964 |
| | Street | 0.977 | 0.974 | 0.969 |
| | Curbstone | 0.991 | 0.734 | 0.995 |
| | Car | 0.960 | 0.905 | 0.977 |
| | Fence | 0.981 | 0.739 | 0.986 |
| SegNet-2 | Background | 0.982 | 0.874 | 0.765 |
| | Street | 0.821 | 0.809 | 0.691 |
| | Curbstone | 0.972 | 0.610 | 0.923 |
| | Car | 0.479 | 0.472 | 0.658 |
| | Fence | 0.831 | 0.566 | 0.857 |
| FCN-8s | Background | 0.965 | 0.963 | 0.867 |
| | Street | 0.974 | 0.967 | 0.926 |
| | Curbstone | 0.984 | 0.532 | 0.913 |
| | Car | 0.893 | 0.527 | 0.609 |
| | Fence | 0.992 | 0.456 | 0.864 |
| U-Net | Background | 0.890 | 0.745 | 0.476 |
| | Street | 0.565 | 0.560 | 0.463 |
| | Curbstone | 0.536 | 0.336 | 0.717 |
| | Car | 0.956 | 0.092 | 0.116 |
| | Fence | 0.038 | 0.028 | 0.128 |

TABLE II. DISTRIBUTION OF THE LABELED CLASSES FOR EXPERIMENTAL DATA

| **Class** | **Number of Pixels** | |
|---|---|---|
| | ***Absolute*** | ***Relative*** |
| Background | 7,842,797 | 29.9% |
| Street | 14,468,900 | 55.2% |
| Barrier | 2,532,555 | 9.7% |
| Car | 1,064,401 | 4.1% |
| Small Obstacle | 305,747 | 1.2% |

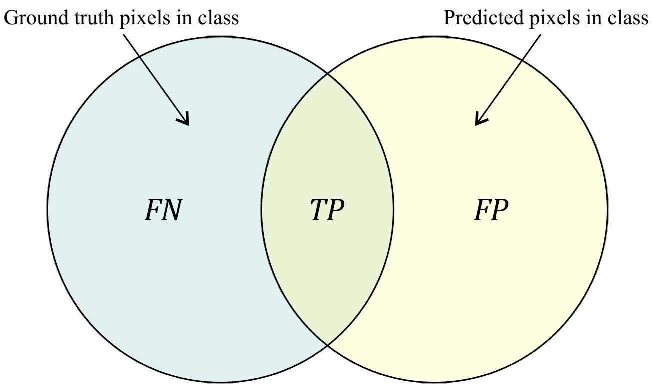


Fig. 8. Visualization of false negatives, true positives, and false positives.

## B. Training

Ideally, all classes should each have the same number of pixels. However, a common issue with semantic segmentation tasks is that the classes in the simulation dataset are imbalanced. To address this issue, we calculate the reciprocal of the relative occurrence frequency $f_k$ of the corresponding class $k = 1, \dots, K$ from the number of all pixels $NP_k$ of the $k$-th class in the entire dataset:

$$f_k = NP_k \cdot \left(\sum_{k=1}^{K} NP_k\right)^{-1} \tag{2}$$

This value is used for the pixel-wise cross entropy loss function $\mathcal{L}$, which is fed by the output $p_k(\boldsymbol{x})$ of the soft-max layer providing the class probability for the $k$-th channel/class at pixel position $\boldsymbol{x}$:

$$\mathcal{L} = -\sum_x f_{l(x)}^{-1} \cdot \log\left(p_{l(x)}(\boldsymbol{x})\right) + \lambda \cdot \sum w_j^2 \tag{3}$$

Here, $l(\boldsymbol{x})$ denotes the true label of the corresponding pixel and $\sum w_j^2$ is the sum over all squared weights of the neural network, which is multiplied by the L2 regularization factor $\lambda$ in order to reduce overfitting. To further reduce overfitting, the dataset is augmented by an augmenter that epoch-wise randomly transforms the input image by scaling, translation, and rotation.

Training is done by the optimization algorithm stochastic gradient descent with momentum (SGDM) for 100 epochs at an initial learn rate of 0.01 for SegNet and 0.0001 for FCN and U-Net. All learning rates are multiplied by 0.5 after every ten epochs. The momentum is chosen as 0.9. The network is calculated on a Linux workstation equipped with a Nvidia Quadro GV100.

## C. Results

We evaluate the classification performance of the neural networks on the test dataset. For this, the two segmentation

TABLE III. NETWORK EVALUATION FOR EXPERIMENTAL DATA

| **Network** | | **Metrics** | | |
|---|---|---|---|---|
| ***Name*** | ***Class*** | ***Acc*** | ***IoU*** | ***BF*** |
| SegNet-1 | Background | 0.910 | 0.858 | 0.783 |
| | Street | 0.956 | 0.933 | 0.852 |
| | Barrier | 0.926 | 0.782 | 0.874 |
| | Car | 0.922 | 0.752 | 0.820 |
| | Small Obtst. | 0.805 | 0.583 | 0.713 |
| SegNet-1 (upper image half) | Background | 0.915 | 0.850 | 0.739 |
| | Street | 0.958 | 0.936 | 0.867 |
| | Barrier | 0.914 | 0.766 | 0.842 |
| | Car | 0.874 | 0.760 | 0.819 |
| | Small Obtst. | 0.712 | 0.528 | 0.701 |
| SegNet-1 (without HGM) | Background | 0.893 | 0.846 | 0.768 |
| | Street | 0.950 | 0.923 | 0.841 |
| | Barrier | 0.903 | 0.750 | 0.841 |
| | Car | 0.891 | 0.726 | 0.795 |
| | Small Obtst. | 0.738 | 0.528 | 0.734 |
| SegNet-2 | Background | 0.870 | 0.789 | 0.692 |
| | Street | 0.917 | 0.891 | 0.733 |
| | Barrier | 0.908 | 0.734 | 0.798 |
| | Car | 0.953 | 0.667 | 0.676 |
| | Small Obtst. | 0.844 | 0.426 | 0.637 |
| FCN-8s | Background | 0.845 | 0.790 | 0.635 |
| | Street | 0.909 | 0.889 | 0.658 |
| | Barrier | 0.919 | 0.696 | 0.709 |
| | Car | 0.945 | 0.588 | 0.482 |
| | Small Obtst. | 0.884 | 0.330 | 0.351 |
| U-Net | Background | 0.355 | 0.275 | 0.322 |
| | Street | 0.801 | 0.640 | 0.249 |
| | Barrier | 0.821 | 0.520 | 0.610 |
| | Car | 0.160 | 0.082 | 0.195 |
| | Small Obtst. | 0.169 | 0.037 | 0.154 |

metrics accuracy $Acc$ and intersection over union $IoU$ are calculated:

$$Acc = \frac{TP}{TP+FN} \tag{4}$$

$$IoU = \frac{TP}{TP+FN+FP} \tag{5}$$

True positives $TP$, false negatives $FN$, and false positives $FP$ correspond to the schema in Fig. 8. In addition, the metric boundary contour matching score $BF$ (also denoted as $F_1$ score) is calculated. This value indicates how well the predicted boundary of each class aligns with the true boundary. Its definition is given in [17].

The calculated metrics are listed in Table I. Accordingly, SegNet-1 performs best, which is also evident when considering the output example in Fig. 7. In particular, the $IoU$ score for SegNet-1 is a surprisingly good result, since the ground truth also covers areas that are not entirely visible to the radar, like the back sides of cars. This is why cars often appear L-shaped inside the radar map. As the simulated street as a non-reflecting surface doesn't provide any radar detections, the greatest difference between the Background and Street classes is the OGM value. However, the OGM normally does not assign the complete street as free space in contrast to our labeled output images. As a consequence, memorizing and overfitting may occur. To overcome this issue, besides more training data, an adapted labeling can be

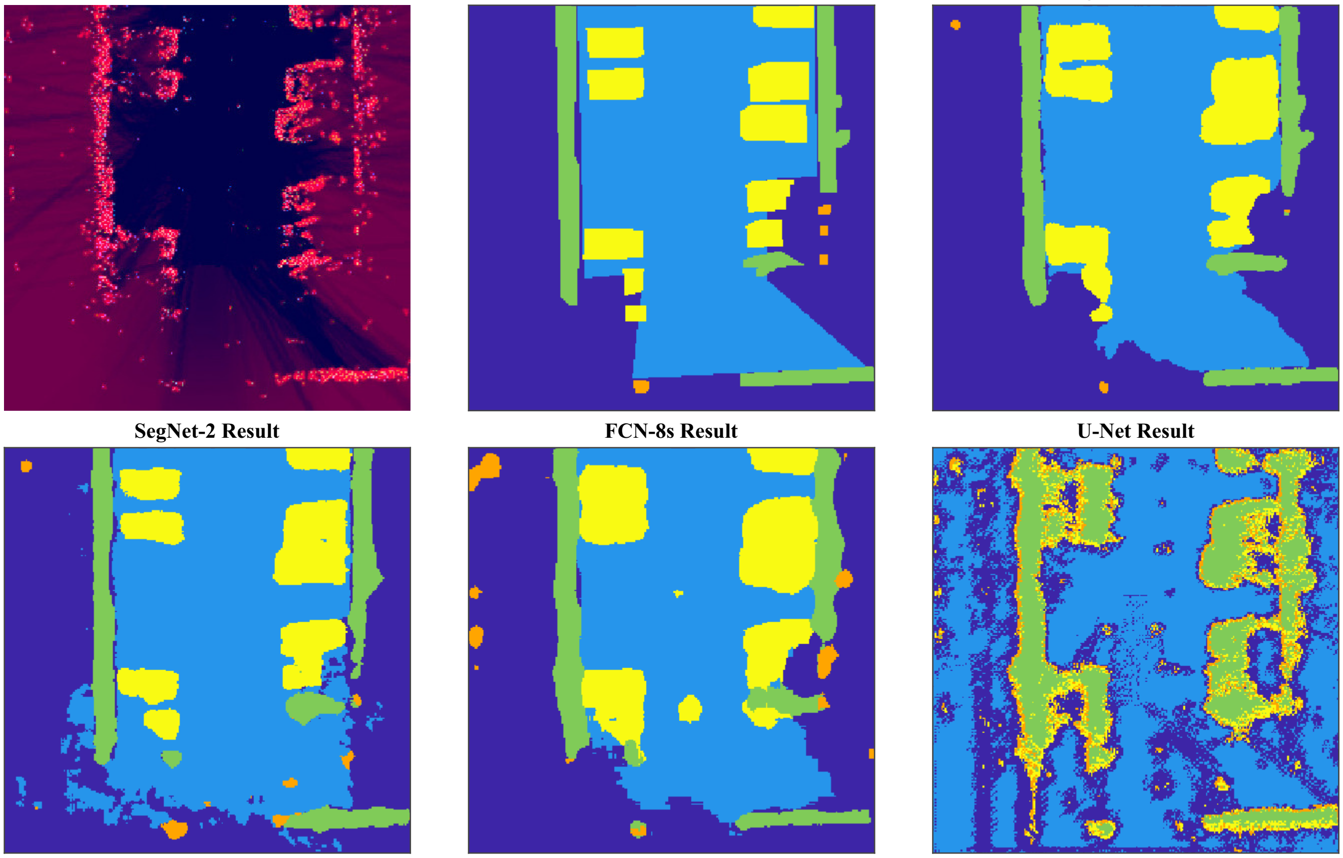


Fig. 9. In this challenging scenario, the input image shows significantly more clutter than the simulated input image in Fig. 7. This results mainly from the reflecting surfaces of the street and roof. However, the OGM assigns unoccupied regions mostly as free space. Consequently, the best performing network – SegNet-1 – yields a very good $IoU$ for the Street class (light blue). Furthermore, in this example the system distinguishes remarkably well between cars (yellow) and barriers (green) to the benefit of automated parking systems (compare [18]). As the transition between clutter belonging to the Background class (dark blue) and small obstacles (orange) is fluently, their distinction is hard for the network.

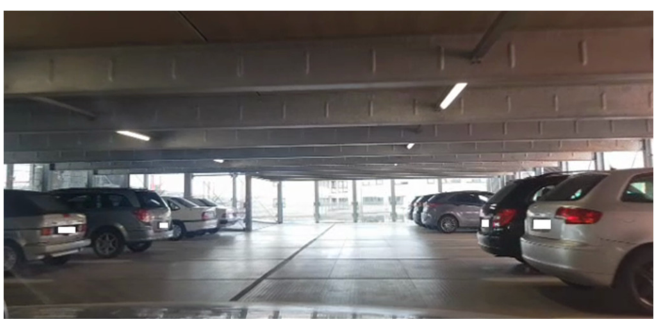

Fig. 10. Real-life scene corresponding to Fig. 9. The lowest roof elements are approx. 2 m high.

performed. Also, the time from one radar map to the next should not be too short in order to avoid too similar images in the dataset.

If SegNet-1 is only trained on the upper image halves (the regions that the test vehicle has passed), the overall results are better, although the number of the training pixels is halved. The other networks achieve better results in this case as well. This corresponds to the general observation that the radar image quality is the better the more measurements are done inside the corresponding region.

The result of SegNet-2 is significantly worse than the one of SegNet-1 – this is not surprising considering its slimmer architecture. However, its performance is clearly better than the performance of U-Net, which obviously struggles handling radar maps, and slightly worse than the performance of FCN-8s. This ranking stays constant when using only the upper image halves.

## V. Experiment

### A. Dataset and Training

The test vehicle conducted four drives through typical urban environments including park decks. Park decks are very challenging scenarios for radar sensors due to the roof and the large amount of metallic infrastructure, which together are a source of many various multipaths for the electromagnetic waves. Consequently, clutter targets occur very frequently.

In contrast to simulation data, the labeling is performed manually according to the ground truth video. The input image size is reduced to 256x256 – which corresponds to 25.6x25.6 m – to increase labeling accuracy (the ground truth video does not cover a larger region). The label pixels are assigned to the five classes Background, Street, Barrier, Car, and Small Obstacle. The latter class addresses not only small obstacles but also regions belonging actually to the other classes when the radar image provides only a few detections of the object.

Altogether, 400 real-world images are used in the same way as the simulation images. The pixels' distribution according to their classes is shown in Table II. There is clearly a marked imbalance between the classes. This is not surprising for this relatively small dataset.

### B. Results

Results are evaluated by processing the test dataset. The metrics are listed in Table III. Once again, SegNet-1 achieves the best metrics – an output example is given in Figs. 9 and 10 – followed by SegNet-2, FCN-8s, and U-Net. If only the upper image halves are used for training, the results will stay approximately constant (the results for the simulation data became better in this case). The results will become slightly worse, if the height information is not available, or, rather, all detections have the same height in the HGM, although the Barrier and the Small Obstacle classes contain objects with different characteristic heights.

In general, the metrics quality of the simulation data is not achieved. This is mainly due to the much more complex environment, which generates situation-dependent clutter targets and no constant false alarms. Also, the Background class is not only the initial value of the radar maps, it also contains a lot of reflections of various unusual objects. Furthermore, defining the optimal number of classes to balance advanced scene understanding with good classification results is challenging. Moreover, as we mentioned in section IV C, the labeled ground truth/the desired output of the neural network must take radar specifications into account. For example, the network performance changes, if a car is labeled as L-shape, U-shape, small rectangle (covers only the radar detections), or big rectangle (covers the complete ground truth). We chose a mix of small and big rectangles.

Inadequately estimating the ego motion between two consecutive sampling loops also degrades the neural-network performance on real-world data. Even if the most sophisticated SLAM algorithm is used, the system cannot accurately ascertain the location of the test vehicle, as defined for simulation data, especially in the case of highly dynamic maneuvers. Additionally, random radar sensor asynchronization, which can be up to 10 ms for test vehicles, leads to further mapping errors.

## VI. Conclusion

In this paper, we presented an algorithm for classifying the stationary infrastructure around a test vehicle by means of semantic segmentation networks. Consequently, the clustering step is not needed anymore. For data generation, the test vehicle was equipped with four low-cost high resolution corner radar sensors, whose radar detections were used to build radar maps as input for the neural network. The algorithm is suitable for use with other radar sensor configurations as well.

For proof of concept, the algorithm was tested on custom simulation as well as real-world data. SegNet, whose architecture was discussed in detail, yielded better results than FCN and U-Net for both datasets. Even our slim SegNet variant, which required only 3 MB RAM for the images of the experimental dataset, beat FCN and U-Net. This shows that the network performance on camera images cannot be directly repeated on radar data. Moreover, the output image labeling is more complicated for radar maps and needs further investigation in future.

Until now, [5] is the only publication that uses semantic segmentation networks for radar maps classification. The results of our algorithm demonstrate that we have set a new benchmark in terms of both the number of different classes and classification accuracy. We have shown that radar sensors provide an advanced stationary scene understanding. Possible applications are parking space identification and lane detection, amongst others. However, more training data are mandatory for better results, greater robustness, and no overfitting. In this context, the radar community should develop a common dataset analogous to camera images. With such a dataset, algorithms from different academic institutions could be compared.

## References


[1] R. Prophet et al., "Pedestrian Classification for 79 GHz Automotive Radar Systems," 29th Intelligent Vehicles Symposium, June 2018.

[2] R. Prophet et al., "Image-Based Pedestrian Classification for 79 GHz Automotive Radar," 15th European Radar Conference, September 2018.

[3] O. Schumann, M. Hahn, J. Dickmann, and C. Wöhler, "Semantic Segmentation on Radar Point Clouds," 21st International Conference on Information Fusion, July 2018.

[4] J. Lombacher, M. Hahn, and J. Dickmann, "Potential of Radar for Static Object using Deep Learning Methods," International Conference on Microwaves for Intelligent Mobility, May 2016.

[5] J. Lombacher et al., "Semantic Radar Grids," 28th Intelligent Vehicles Symposium, June 2017.

[6] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," Conference on Computer Vision and Pattern Recognition, June 2015.

[7] V. Badrinarayanan, A. Kendall, and Roberto Cipolla, "SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation," Transactions on Pattern Analysis and Machine Intelligence, vol. 39, Issue 12, December 2017.

[8] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Miomedical Image Segmentation," Medical Image Computing and Computer-Assisted Intervention, October 2015.

[9] C. Sturm et al., "Automotive Fast-Chirp MIMO Radar with Simultaneous Transmission in a Doppler-Multiplex," 19th International Radar Symposium, June 2018.

[10] R. Schubert et al., "Empirical Evaluation of Vehicular Models for Ego Motion Estimation," 22th Intelligent Vehicles Symposium, June 2011.

[11] M. Schoen, M. Horn, M. Hahn, and J. Dickmann, "Real-Time Radar SLAM," 11. Workshop Fahrerassistenzsysteme und automatisiertes Fahren, March 2017.

[12] H. Moravec and A. E. Elfes, "High Resolution Maps from Wide Angle Sonar," International Conference on Robotics and Automation, March 1985.

[13] R. Prophet et al., "Adaptions for Automotive Radar Based Occupancy Gridmaps," International Conference on Microwaves for Intelligent Mobility, April 2018.

[14] A. Hornung et al., "OctoMap: an efficient probabilistic 3D mapping framework based on octrees," Autonomous Robots, vol. 34, Issue 3, February 2013.

[15] L.-C. Chen et al., "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," Transactions on Pattern Analysis and Machine Intelligence, vol. 40, Issue 4, April 2018.

[16] K. Simonyan and A. Zissermann, "Very deep convolution networks for large-scale image recognition," International Conference on Learning Representations, April 2015.

[17] G. Scurka, D. Larlus, and F. Perronnin, "What is a good evaluation measure for semantic segmentation?," British Machine Vision Conference, September 2013.

[18] R. Prophet et al., "Parking Space Detection from a Radar Based Target List," International Conference on Microwaves for Intelligent Mobility, March 2017.